



INTERNAL — OPERATIONS

QynSight Monitoring Runbook (Native Engine)

Qyenyx vOPS HUB — v1.0.0 RC1 · Document v2.0

Document Version	2.0
Software Version	v1.0.0 RC1
Applies To	Qyenyx vOPS HUB v1.0.0 RC1
Classification	Internal — Operations
Owner	Operations / SRE
Status	Released
Last Updated	2026-06-29
Document Type	Operations runbook

REVISION HISTORY

Version	Date	Notes
1.0	2026	"ShieldObserve / Approach 2" Nagios integration runbook.
2.0	2026-06-29	Rewritten for the **native** QynSight monitoring engine. Nagios procedures moved to Appendix A (legacy migration / optional integration only).

Table of Contents

1. QynSight monitoring architecture	0
2. Prerequisites	0
3. Collectors and check types	0
4. Day-to-day operations	0
4.1 Define monitored targets (Discovery)	0
4.2 Run checks (Monitoring Engine)	0
4.3 Evaluate alerts (Alert Engine)	0
4.4 Read monitoring data (Analytics / Capacity / Infra Map)	0
5. Health verification (end-to-end, workspace 84)	0
6. Troubleshooting	0
"Last run: never" / stale data	0
Port scan never completes	0
A check is stuck / always Unknown	0
Workspace data leakage	0
Validation errors on PUT /observe/targets (422)	0
Where to look	0
7. Database reference	0
observe_services schema (real column names)	0
8. Frontend (use the real API)	0
9. Build / test verification	0
Appendix A — Legacy Nagios (migration / optional integration only)	0

QynSight Monitoring Runbook (Native Engine)

This runbook covers operating and troubleshooting **QynSight**, Quenyx's **native** monitoring stack. Monitoring runs **inside the platform** — there is **no external monitoring daemon and no Nagios dependency**. Service checks (HTTP, TCP, Ping, and custom plugin scripts) execute in PHP on the Laravel scheduler and write results directly to `observe_services` (`engine_key = 'native'`), which the Services / Targets / Infrastructure Map UI reads.

Nagios is not part of the runtime. It may appear **only** as a legacy migration source or as an optional third-party integration (Integrations page). See **Appendix A** if you are operating a legacy Nagios deployment during migration.

1. QynSight monitoring architecture

QynSight is composed of native engines:

Engine	Responsibility	Operational surface
Discovery Engine	Find hosts/services; port scans	<code>observe/targets</code> , <code>targets/validate</code> , per-host port scan (queue worker)
Monitoring Engine	Run due service checks in-platform	<code>php artisan observe:run-checks</code> (scheduled)
Metrics Engine	Ingest agent metrics / inventory / heartbeats	agent token endpoints → <code>agent_metrics</code> , summaries
Service Checks	Per-service definitions + intervals	<code>observe/services</code> , <code>observe/service-definitions</code>
Alert Engine	Evaluate rules → events → channels	<code>php artisan observe:evaluate-alerts</code> (scheduled), <code>alerts/*</code>
Capacity Planning	Trend / forecast over metrics	<code>observe/reports</code> , capacity views
Analytics	Performance analytics & reporting	<code>reports/summary</code> , performance views
Infrastructure Map	Live host/service topology	infra topology endpoints

Check result model: every check maps to the standard exit-code convention — `0 = OK` , `1 = Warning` , `2 = Critical` , `3 = Unknown` — and upserts a row into `observe_services` with `state` , `last_check_at` , `next_check_at` , `output` , and `perfdata` .

2. Prerequisites

1. **Run migrations** (creates the `observe_*` and `agents*` tables):

```
cd backend
php artisan migrate --force
```

Required tables include `observe_targets_hosts`, `observe_targets_services`, `observe_services`, `observe_meta`, plus the `observe_alert*` and monitoring-profile tables. If you see "Table doesn't exist", migrations have not been run.

2. **Laravel scheduler running** (this is what drives native monitoring):

```
* * * * * cd /var/www/quenyx/quenyx-saas/backend && php artisan schedule:run >> /var/www/quer
```

The scheduler runs `observe:run-checks` (every two minutes, due services only) and `observe:evaluate-alerts` (every minute). See `app/Console/Kernel.php`.

3. **Queue worker running** (used by port scans / discovery jobs):

```
cd backend && php artisan queue:work --queue=default --sleep=1 --tries=3
```

4. **Plugins directory** (only if you use custom plugin checks):

```
cd backend && php artisan observe:install-plugins
```

Plugins live under `storage/app/observe_plugins` (override with `OBSERVE_PLUGINS_DIR`). Plugin execution timeout is `config('observe.plugin_timeout_seconds')`.

No Docker/Nagios containers are required for native monitoring. Do **not** start a Nagios daemon for the native engine.

3. Collectors and check types

Native checks supported by the Monitoring Engine:

- **HTTP** — PHP HTTP client to `host:port/path`; compares status to expected (e.g. 200).
- **TCP** — PHP socket connect to `host:port`.
- **Ping** — system `ping` (or socket); parses RTA / loss.
- **Plugin** — custom PHP / Perl / shell script in the plugins dir, selected via a service of type `plugin` with `check_args.plugin = <script>`. The engine passes context as environment variables:

Variable	Description
<code>OBSERVE_HOST_ADDRESS</code>	Host IP / hostname of the monitored target
<code>OBSERVE_HOST_NAME</code>	Scoped host name (e.g. <code>ws84-MyHost</code>)
<code>OBSERVE_SERVICE_NAME</code>	Service name (e.g. <code>Disk /</code>)
<code>OBSERVE_WORKSPACE_ID</code>	Workspace ID
<code>OBSERVE_CHECK_ARGS</code>	JSON object of <code>check_args</code>

The plugin's **exit code** maps to state (0/1/2/3) and its **stdout** first line becomes the check output; append `| perfdata` for performance data. Only scripts inside the `plugins` dir run (path traversal is blocked). See `backend/docs/OBSERVE_NATIVE_CHECKS.md` for full plugin authoring detail.

4. Day-to-day operations

4.1 Define monitored targets (Discovery)

Targets (hosts + their services) are managed per workspace via the API/UI. Host names are scoped with a `ws{id}-` prefix for tenant isolation.

```
# Auth (adjust credentials)
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"password"}' | jq -r '.data.token')

# Define hosts + services for workspace 84
curl -s -X PUT http://127.0.0.1:8000/api/workspaces/84/observe/targets \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"hosts":[{"name":"web-server-01","address":"192.168.1.10","enabled":true,
    "services":[{"name":"HTTP","check_command":"check_http","enabled":true},
      {"name":"Ping","check_command":"check_ping","enabled":true}]}]}' | jq
```

4.2 Run checks (Monitoring Engine)

The scheduler runs checks automatically. To run on demand:

```
cd backend
# Run all due checks for one workspace
php artisan observe:run-checks --workspace_id=84
# Run due checks across all workspaces (what the scheduler does)
php artisan observe:run-checks
```

Verify results were written:

```
php artisan tinker --execute="echo \App\Models\ObserveService::where('workspace_id',84)→where('"
```

4.3 Evaluate alerts (Alert Engine)

```
cd backend
php artisan observe:evaluate-alerts --workspace_id=84
```

Review alert events and channels via the API:

```
curl -s -H "Authorization: Bearer $TOKEN" "http://127.0.0.1:8000/api/workspaces/84/alerts/summary"
curl -s -H "Authorization: Bearer $TOKEN" "http://127.0.0.1:8000/api/workspaces/84/alerts/history"
```

4.4 Read monitoring data (Analytics / Capacity / Infra Map)

```
# Service state (only this workspace's data)
curl -s -H "Authorization: Bearer $TOKEN" "http://127.0.0.1:8000/api/workspaces/84/observe/services"
# Summary / health
curl -s -H "Authorization: Bearer $TOKEN" "http://127.0.0.1:8000/api/workspaces/84/observe/summary"
# Reports (analytics / capacity)
curl -s -H "Authorization: Bearer $TOKEN" "http://127.0.0.1:8000/api/workspaces/84/observe/reports"
```

5. Health verification (end-to-end, workspace 84)

```
# 1. Targets defined
php artisan tinker --execute="echo \App\Models\ObserveTargetHost::where('workspace_id',84)→count()"

# 2. Run checks
php artisan observe:run-checks --workspace_id=84

# 3. Native rows present
php artisan tinker --execute="echo \App\Models\ObserveService::where('workspace_id',84)→where('engine_key','native')→count()"

# 4. API returns scoped rows
curl -s -H "Authorization: Bearer $TOKEN" "http://127.0.0.1:8000/api/workspaces/84/observe/services?workspace_id=84"

# 5. Last run recorded
php artisan tinker --execute="dump(\App\Models\ObserveMeta::where('workspace_id',84)→first(['updated_at']))"

# 6. UI: open /observe/services for workspace 84 and confirm rows render.
```

Checklist

- ☐ Scheduler cron active (`schedule:run` every minute) — drives `observe:run-checks`.
- ☐ Queue worker active (port scans / discovery jobs).
- ☐ Targets defined for the workspace (`ws{id}-` prefixed host names).
- ☐ `observe:run-checks` writes `observe_services` rows with `engine_key='native'`.
- ☐ `observe:evaluate-alerts` produces alert events when thresholds are crossed.

- ☐ API returns **only** the workspace's services (tenant isolation).

6. Troubleshooting

"Last run: never" / stale data

The Laravel **scheduler is not running**. Confirm the cron entry for `php artisan schedule:run` exists and is firing; check `storage/logs/scheduler.log`. Run `php artisan observe:run-checks` once manually to confirm checks execute.

Port scan never completes

The **queue worker** is not running. Start `php artisan queue:work` and re-trigger the scan.

A check is stuck / always Unknown

- Confirm the target host `address` is reachable from the app host.
- For plugin checks: confirm the script exists in the plugins dir, is executable, and returns a valid exit code (0/1/2/3) within `observe.plugin_timeout_seconds`. Check `storage/logs/laravel.log` for the engine's stderr capture.

Workspace data leakage

All native host names must be `ws{id}-` prefixed and all queries are workspace-scoped:

```
php artisan tinker --execute="dump(\App\Models\ObserveService::where('workspace_id',84)→pluck('
```

Every value must start with `ws84-`.

Validation errors on `PUT /observe/targets` (422)

Names are sanitized to `[A-Za-z0-9_-]` (spaces → hyphens); check for duplicates after sanitization and ensure `check_command` is one the engine supports (HTTP/TCP/Ping/plugin).

Where to look

- **Backend:** `storage/logs/laravel.log` (check execution, alert evaluation), `storage/logs/scheduler.log`.
- **Audit:** sensitive monitoring actions are recorded in `audit_logs`.

7. Database reference

Observe tables use **plural** target table names:

- `observe_targets_hosts` — workspace-defined hosts (`ObserveTargetHost`)
- `observe_targets_services` — workspace-defined services on hosts (`ObserveTargetService`)

- `observe_services` — check-result state (`ObserveService`)
- `observe_meta` — last run/state per workspace/engine (`ObserveMeta`)
- `observe_alert*` / monitoring profile tables — alert rules, events, channels, profiles

`observe_services` schema (real column names)

The engine and API use `service_name` (not `service_description`).

Field	Type	Null	Key	Default	Extra
id	bigint unsigned	NO	PRI	NULL	auto_increment
workspace_id	bigint unsigned	NO	MUL	NULL	
engine_key	varchar(50)	NO		nagios	(native checks write <code>native</code>)
engine_service_key	varchar(255)	NO		NULL	
host_name	varchar(255)	NO		NULL	
service_name	varchar(255)	NO		NULL	
state	varchar(20)	NO		NULL	
last_check_at	datetime	YES		NULL	
duration_sec	int	YES		NULL	
attempt	varchar(255)	YES		NULL	
output	text	YES		NULL	
perfdata	text	YES		NULL	
created_at	timestamp	YES		NULL	
updated_at	timestamp	YES		NULL	

Note (implementation detail): the `engine_key` column default in the original migration is `nagios` for backward compatibility, but the **native** engine writes `engine_key = 'native'` . When both engines are present, native data is preferred/deduped in the UI.

8. Frontend (use the real API)

The Observe UI must call the real backend, not fixtures.

```
cd frontend
# Ensure fixtures are off
echo "VITE_OBSERVE_USE_FIXTURES=false" >> .env.production
npm run build
# Deploy dist/ to your static host / reverse proxy
```


After deploy, open `/app/workspaces/84/observe/services` ; it should show real rows after `observe:run-checks` runs. If it shows "No services found" while the DB has rows, the build likely used `VITE_OBSERVE_USE_FIXTURES=true` or the backend is unreachable.

9. Build / test verification

```
# Backend
cd backend && php artisan test --filter=Observe

# Frontend (real Observe API)
cd frontend && VITE_OBSERVE_USE_FIXTURES=false npm run build

# Gateway (edge/proxy only; not required for native monitoring)
cd gateway && npm run build
```

Appendix A — Legacy Nagios (migration / optional integration only)

Not a platform dependency. The platform monitors natively. This appendix is retained **only** for teams migrating from, or optionally integrating, an external Nagios deployment. New environments should not enable it.

When a legacy Nagios deployment is in use, an optional gateway path can publish workspace configs to Nagios and poll its status:

- **Publish:** `php artisan observe:nagios:publish --workspace_id=84` writes a workspace config (`ws{id}-` prefixed hosts) that Nagios loads via `cfg_dir=/opt/nagios/etc/objects/quenyx/workspaces` in the main `nagios.cfg` (never via `cfg_file=.../quenyx.cfg`).
- **Reload:** `POST {gateway}/internal/engines/nagios/reload` (or `docker exec nagios-core kill -HUP 1`).
- **Poll:** the legacy `observe:poll` command reads Nagios status into `observe_services` (`engine_key='nagios'`). **This command is disabled in the scheduler by default** (see `app/Console/Kernel.php`); native checks are the production path.
- **Toggle:** `OBSERVE_AUTO_PUBLISH_NAGIOS` and the gateway `NAGIOS_*` env vars (`NAGIOS_CONFIG_DIR` , `NAGIOS_CONTAINER_NAME` , `NAGIOS_BASE_URL` , `NAGIOS_USER` , `NAGIOS_PASS`) configure this optional path.

If you are not migrating from Nagios, ignore this appendix entirely and rely on `observe:run-checks` + `observe:evaluate-alerts` .